

COMPUTER SCIENCE TRIPOS Part IB 75%, Part II 50% – 2020 – Paper 7

10 Prolog (acr31)

In this question we will write a program to assign guests to seats around a dinner table. The table is rectangular with the same number of seats along both of the long sides. There are no seats along the short sides. Two guests sit *near* each other if they occupy adjacent seats on the same side of the table, or sit directly opposite one another. Each guest at the dinner table may express a preference to sit near to one or more other guests.

When answering this question you should ensure that each of your predicates has a comment giving a declarative reading of its behaviour and you should avoid unnecessary use of cut. Your solutions should not use any extra-logical predicates (such as assertz) and you should not assume the existence of any library predicates.

- generate and test (a) Define the concept of *generate and test* using this problem as an example. [2 marks]

Answer: In generate and test we first *generate* a candidate solution and we then *test* to see if it meets the constraints (1 mark). In the context of this problem we can generate a seating assignment and then test that it meets the preferences (1 mark).

- terms and data representation (b) Describe a suitable approach for representing guests, their places at the table and their preferences. [2 marks]

Answer: Any workable scheme is acceptable. One example would be to store two lists, one for each side of the table. Each position in the list is a seat. People are indicated with numbers and preferences are pairs (A,B) indicating that guest A wishes to be near guest B. We'll store the preferences as a list of pairs. Storing two lists is helpful because it makes it easier later on to work out if one person is opposite the other.

- facts, rules and backtracking (c) Implement a predicate `nextTo(A,B,Assigned)` which is true if guest A is sitting adjacent-to or directly opposite guest B in the seating assignment `Assigned`. [4 marks]

Answer: Answers should make use of the scheme outlined above. Candidates may use extra arguments for their data structure if they choose. For example:

```
% nextTo(A,B,L,R) succeeds if people A and B are either adjacent or opposite
% in the seating assignment L,R
nextTo(A,B,[A,B|_],_).
nextTo(A,B,_,[A,B|_]).
nextTo(A,B,[B,A|_],_).
nextTo(A,B,_,[B,A|_]).
nextTo(A,B,[A|_],[B|_]).
nextTo(A,B,[B|_],[A|_]).
nextTo(A,B,[_|T1],[_|T2]) :- nextTo(A,B,T1,T2).
```

1) Succeeding if A is adjacent to B (1 mark); 2) Succeeding if A is opposite B (1 mark); 3) Symmetric relation (1 mark); 4) Declarative comment (1 mark)

facts, rules and
backtracking

- (d) Implement a predicate `satisfied(Assigned, Prefs)` which is true if `Assigned` is a seating assignment which meets all the preferences specified in `Prefs`.

[3 marks]

Answer:

```
% satisfied(L,R,P) is true if (L,R) is a seating assignment which meets all
% the preferences in the list P
satisfied(_,_, []).
satisfied(L,R,[(A,B)|T]) :- nextTo(A,B,L,R), satisfied(L,R,T).
```

1) Correctly checking all the preferences (1 mark); 2) Only one result on backtracking (1 mark); 3) Declarative comment (1 mark)

generate and test

- (e) Implement a predicate `assign(N, Prefs, Assigned)` which succeeds if `Assigned` is a valid seating assignment for `N` guests with preferences `Prefs`. Your predicate should be able to generate the assignment and should fail if an odd number of guests are requested.

[6 marks]

Answer:

```
% gen(N,L) generates a list of integers from N down to 1 (inclusive)
gen(0, []) :- !.
gen(N, [N|R]) :- M is N-1, gen(M,R).
```

```
% take(L,E,R) succeeds if E is an element taken from L and R is left over
take([H|T], H, T).
take([H|T], R, [H|S]) :- take(T, R, S).
```

```
% perm(L1,L2) succeeds if L2 is a permutation of the list L1
perm([], []).
perm(L, [H|T]) :- take(L, H, R), perm(R, T).
```

```
% split(List,L,R) splits List in half and succeeds if L contains all the odd
% indexed elements of List and R contains all the even indexed elements
split([], [], []).
split([X,Y|T], [X|X1], [Y|Y1]) :- split(T, X1, Y1).
```

```
% assign(N,Prefs,Ass) succeeds if Ass is a seating assignment which meets
% the preferences in Prefs for N people
assign(N,X,(L1,R1)) :- gen(N,A),
                        perm(A,A1),
                        split(A1,L1,R1),
                        satisfied(L1,R1,X).
```

1) Declarative comments (2 marks); 2) Correctly generating all possible assignments on backtracking (2 marks); 3) Testing each possible assignment (1 mark); 4) Generates other possible assignments when backtracking (1 mark)

iterative
deepening

- (f) What technique could you use to find the best assignment in those cases where it is not possible to meet the preferences of all the guests? Explain in words how you would alter your solution to do this.

[3 marks]

Answer: You could use iterative deepening (1 mark). Rather than marking an assignment valid you could score it as the number of preferences it fails to meet (1 mark). Then you would change the assign predicate to first look for an assignment with score of 0 then with score of 1 and so on (1 mark).
